

Curs 5 — Structuri de Date (Gabriel Istrate)

Sinteză: Arbori binari de căutare (Binary Search Trees)

🎯 = punct de examen, ⚠️ = capcană, ★ = fapt fundamental de memorat.
Cursul începe cu coada hashing-ului (scorecard-ul probing-ului) și apoi intră în BST — subiectul cel mai testat din toată materia.

0. De ce e cursul cel mai important pentru grilă

BST-urile sunt **topul absolut** al întrebărilor de examen. Practic toate astea ies direct din acest curs:

Tip de întrebare	Unde apare	Ce-ți trebuie
Reconstrucție din parcurgeri (pre↔in↔post)	grila2025 Q2, grila2023 Q3, model Q2	inordine = sortat crescător
Al k-lea element / median	grila2025 Q13, model Q12, grila2023 Q13	inordine dă ordinea sortată
Listare descrescătoare	grila2023 Q15	reverse-order = R-V-L
Evaluarea unui arbore de expresie	grila2023 Q22	postordine
Inserare secvență → înălțime	grila2023 Q5	input sortat → înălțime $O(n)$
Inserare secvență → postordine	grila2023 Q25	construiești arborele, apoi postordine
Ștergere prin copiere (delete by copy)	grila2025 Q3, model Q3	succesor + copiere
Proprietăți succesor	model Q3	succesor = frunză sau fără copil stâng
Rotații (rotește X în jurul lui Y)	model Q7, grila2023 Q10	invariantul rotației
Garanție $O(\log n)$ worst-case	grila2025 Q17	BST simplu NU garantează; balansat DA

1. Coada hashing-ului: scorecard-ul probing-ului + tranziția

Scorecard hash tables (complexități medii ale căutării, reușită / nereușită):

Metodă	Complexitate
INSERT/SEARCH/DELETE, chaining	$O(1 + \alpha)$
SEARCH, linear probing	$\frac{1}{2}(1 + 1/(1-\alpha))$, $\frac{1}{2}(1 + 1/(1-\alpha)^2)$
SEARCH, quadratic probing	$1 - \ln(1-\alpha) - \alpha/2$, $1/(1-\alpha) - \alpha - \ln(1-\alpha)$
SEARCH, double hashing	$(1/\alpha) \cdot \ln(1/(1-\alpha))$, $1/(1-\alpha)$

⚠ Aceste formule sunt **de referință** (Drozdek/Knuth) — NU le memora; reține doar ideea: cu cât α e mai mare (tabelul mai plin), cu atât mai scump, iar double hashing e cel mai bun. Cursul însuși zice că sunt greu de analizat matematic.

Tranziția: „Poate $O(1+\alpha)$ e prea ambițios? Ceva de tipul $O(\log n)$? În practică $\log(n)$ e un număr mic!” → de aici pornesc arborii binari de căutare ca alternativă la hashing.

2. BST: definiție, interfață, structură

Un arbore binar de căutare implementează o **mulțime dinamică** peste un **domeniu total ordonat**.

Interfața (toate operațiile de mulțime dinamică din Cursul 2):

- `Tree-Insert(T, k)`, `Tree-Delete(T, k)`, `Tree-Search(T, x)`;
- parcurgeri (tree-walks): `Inorder-Tree-Walk(T)` etc.;
- `Tree-Minimum(T)`, `Tree-Maximum(T)`;
- iterare: `Tree-Successor(x)`, `Tree-Predecessor(x)`.

Structura:

- `T.root` = rădăcina arborelui;
- fiecare nod `x` are: `x.parent`, `x.key`, `x.left`, `x.right`.

★ Proprietatea BST (binary-search-tree property)

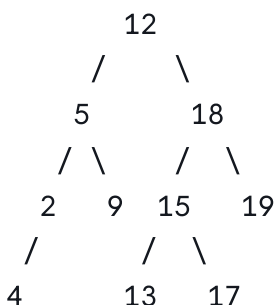
Pentru orice noduri x, y, z :

- $y \in \text{subarborele_stâng}(x) \Rightarrow y.\text{key} \leq x.\text{key}$

- $z \in \text{subarborele_drept}(x) \Rightarrow z.\text{key} \geq x.\text{key}$

Adică: tot ce e în stânga unui nod e $\leq$ el, tot ce e în dreapta e $\geq$ el. Asta e proprietatea din care decurge **absolut tot** restul cursului.

Arborele-exemplu folosit peste tot (memorează-l, apare în multe figuri):



3. Parcurgeri (tree walks) — critic de examen

3.1 Cele patru parcurgeri

Parcursere	Ordine	Pseudocod (recursiv)	Pe arborele-exemplu
Inordine	Stânga, Vârf, Dreapta	$\text{walk(left); print;}$ walk(right)	2 4 5 9 12 13 15 17 18 19
Preordine	Vârf, Stânga, Dreapta	$\text{print; walk(left);}$ walk(right)	12 5 2 4 9 18 15 13 17 19
Postordine	Stânga, Dreapta, Vârf	$\text{walk(left); walk(right);}$ print	4 2 9 5 13 17 15 19 18 12
Reverse-order	Dreapta, Vârf, Stânga	$\text{walk(right); print;}$ walk(left)	19 18 17 15 13 12 9 5 4 2

<pre>Inorder-Tree-Walk(x): if x ≠ nil Inorder-Tree-Walk(x.left) print x.key Inorder-Tree-Walk(x.right)</pre>	<pre>Preorder-Tree-Walk(x): if x ≠ nil print x.key Preorder-Tree-Walk(x.left) Preorder-Tree-Walk(x.right)</pre>
<pre>Postorder-Tree-Walk(x): if x ≠ nil Postorder-Tree-Walk(x.left) Postorder-Tree-Walk(x.right) print x.key</pre>	<pre>Reverse-Order-Tree-Walk(x): if x ≠ nil Reverse-Order-Tree-Walk(x.right) print x.key Reverse-Order-Tree-Walk(x.left)</pre>

(Plus o procedură „starter” care apelează parcurgerea pe $T.root$.)

★ Cele două fapte de aur

- 🌀 **Inordine = elementele în ordine crescătoare (sortate).** De aici ies TOATE întrebările cu „al k-lea element”, „medianul”, „reconstrucție din parcurgeri”.
- 🌀 **Reverse-order (Dreapta-Vârf-Stânga) = ordine descrescătoare.** Asta e răspunsul la grila2023 Q15 („listare în ordine descrescătoare”).

3.2 Aplicația postordinii: evaluarea expresiilor aritmetice

🌀 O expresie aritmetică se reprezintă ca **arbore sintactic**; fiecare nod are un câmp $value$. **Parcurgerea în postordine tipărește notația postfixată și calculează valoarea** (copiii înaintea părintelui = ai operanzii înainte de operator). Exact întrebarea grila2023 Q22 (ordinea de evaluare $a, b, +, c, *, 7, root(+)$).

3.3 Complexitatea parcurgerilor

- Recurența generală: $T(n) = T(n_L) + T(n - n_L - 1) + \Theta(1)$, unde n_L = mărimea subarborelui stâng.
- Toate cele patru parcurgeri: $\Theta(n)$.
- **Putem mai bine? NU** — lungimea ieșirii e $\Theta(n)$ (tipărești toate cele n chei), deci nu poți coborî sub $\Theta(n)$.

4. Minim și Maxim

Din proprietatea BST: **minimul e cel mai din stânga nod, maximul e cel mai din dreapta.**

```

Tree-Minimum(x):
    while x.left != nil
        x = x.left
    return x

Tree-Maximum(x):
    while x.right != nil
        x = x.right
    return x

```

Complexitate: $\Theta(\text{înălțime})$.

5. Succesor și Predecesor — critic de examen

Succesorul lui x = nodul cu următoarea cheie ca valoare. Două cazuri:

1. **Dacă x are subarbore drept:** succesorul = **minimul subarborelui drept** al lui x (mergi o dată dreapta, apoi tot stânga).
2. **Dacă x NU are subarbore drept:** succesorul = **primul strămoș a al lui x astfel încât x se află în subarboarele stâng al lui a** (urci până cobori dintr-un copil stâng).

```

Tree-Successor(x):
    1 if x.right != nil
    2     return Tree-Minimum(x.right)
    3 y = x.parent
    4 while y != nil and x == y.right
    5     x = y
    6     y = y.parent
    7 return y

```

Exemple pe arborele-exemplu:

- **succ(12):** 12 are subarbore drept $\rightarrow$ min(subarbore 18) = mergi la 18, apoi stânga la 15, stânga la 13 $\rightarrow$ **13**.
- **succ(9):** 9 nu are subarbore drept $\rightarrow$ urci: 9 e copil drept al lui 5 (continui), 5 e copil stâng al lui 12 (stop) $\rightarrow$ **12**.

★ Faptul de aur pentru ștergere

🎯 Când x are subarbore drept, succesorul = minimul acelui subarbore = **cel mai din stânga nod al lui** $\rightarrow$ **nu are copil stâng** (e fie frunză, fie are doar copil drept). Asta e exact răspunsul la model Q3: „succesorul e întotdeauna fie o frunză, fie un nod fără copil stâng”.

⚠️ NU „fără copil drept” — capcană clasică. Și e motivul pentru care ștergerea succesorului e ușoară (are cel mult un copil).

6. Căutare (Search)

Căutare binară (de aici și numele arborelui):

```
Tree-Search(x, k):
    if x = nil or k = x.key
        return x
    if k < x.key
        return Tree-Search(x.left, k)
    else return Tree-Search(x.right, k)

Iterative-Tree-Search(T, k):
    x = T.root
    while x ≠ nil and k ≠ x.key
        if k < x.key
            x = x.left
        else x = x.right
    return x
```

- **Corectă** datorită proprietății BST (la fiecare pas elimini jumătate din arbore: dacă $k < x.key$ mergi stânga, altfel dreapta).
- Complexitate: $\Theta(\text{adâncimea arborelui}) = O(n)$ în cazul cel mai rău, $O(\log n)$ în medie.

7. Inserare (Insertion) — critic de examen

Ideea: ca să inserezi x, **cauți** x (mai exact $x.key$); dacă nu-l găsești, îl adaugi **fix acolo unde s-a oprit căutarea** (ca frunză).

```
Tree-Insert(T, z):
    1 y = nil
    2 x = T.root
    3 while x ≠ nil           // coboară până la nil, ținând minte părintele y
    4     y = x
    5     if z.key < x.key
    6         x = x.left
    7     else x = x.right
    8 z.parent = y
    9 if y = nil
    10     T.root = z         // arbore gol
    11 else if z.key < y.key
    12     y.left = z
    13 else y.right = z
```

Complexitate: $\Theta(h)$ (h = înălțimea arborelui).

★ Observația esențială despre înălțime

Atât inserarea cât și căutarea au complexitate **h = înălțimea arborelui**, iar:

- 🌀 $h = O(\log n)$ în medie (cu ordine de inserare aleatoare);
- 🌀 $h = O(n)$ în cazuri particulare — în special cu secvențe deja sortate (arborele degenerază într-un lanț);
- problema: cazul „defavorabil” **nu e deloc rar** (input-uri sortate apar des în practică).

Asta explică grila2023 Q5: inserarea lui 1, 3, 5, 10, 15, 12, 16 (aproape sortat) produce un arbore înalt, dezechilibrat (înălțime 5, lanț spre dreapta).

Ideea de soluție: folosește randomizarea ca să transformi orice caz în cazul mediu.

8. Inserare randomizată (cum garantăm $O(\log n)$ în medie)

- **Idea 1:** inserează orice secvență ca pe o permutare aleatoare a ei. ⚠️ Problemă: secvența A nu e neapărat cunoscută în avans.
- **Idea 2:** obții o permutare aleatoare alternând aleator două proceduri de inserare:
 - **tail insertion** = exact ce face `Tree-Insert` (adaugă jos, ca frunză);
 - **head insertion** = `Tree-Root-Insert` (inserează ca și cum z ar fi fost primul element → ajunge rădăcină).

```
Tree-Randomized-Insert(t, z):
1 if t = nil
2   return z
3 r = valoare aleatoare uniformă din {1, ..., t.size + 1}
4 if r = 1 // Pr[r=1] = 1/(t.size+1)
5   z.size = t.size + 1
6   return Tree-Root-Insert(t, z)
7 if z.key < t.key
8   t.left = Tree-Randomized-Insert(t.left, z)
9 else t.right = Tree-Randomized-Insert(t.right, z)
10 t.size = t.size + 1
11 return t
```

De ce funcționează (slide-ul zice doar „pare că merge” — iată motivul): la fiecare nivel, noul element z devine rădăcina subarborelui curent cu probabilitate exact $1/(size+1)$ — adică fix probabilitatea ca z să fie primul într-o permutare aleatoare a elementelor acelui subarbore. Aplicat recursiv, **fiecare permutare devine egal de probabilă**, deci arborele e distribuit ca și cum ar fi construit dintr-o permutare aleatoare → înălțime așteptată $O(\log n)$, indiferent de ordinea reală a input-ului. (Versiunea ne-recursivă din slide-uri NU merge: ultimul element poate ajunge doar sus sau jos, nu uniform.)

9. Rotații (Rotation) — critic de examen

Rotația rearanjează local trei subarbori păstrând proprietatea BST. Pe configurația cu rădăcina b , copil stâng a , și subarborii $(k \leq a)$, $(a \leq k \leq b)$, $(k \geq b)$:

Right-Rotate(x):	Left-Rotate(x):
1 l = x.left	1 r = x.right
2 x.left = l.right	2 x.right = r.left
3 l.right = x	3 r.left = x
4 return l	4 return r

Right-Rotate urcă copilul stâng (devine noua rădăcină), **Left-Rotate** e inversul.

★ Invariantul rotației (ce trebuie să știi la examen)

- 🌀 **Păstrează proprietatea BST** (parcursul în inordine rămâne neschimbată — rotația doar rebalansează, nu reordonează);
- 🌀 **Păstrează mulțimea de noduri** din subarbore (aceleași noduri, altă formă);
- 🌀 **Schimbă rădăcina** subarborelui;
- 🌀 e o operație **O(1)** (doar câteva repointări).

Asta e exact ce se testează la model Q7 / grila2023 Q10 („rotește 6 în jurul lui 3"): după rotație, anumite noduri devin copii ai altora, **mărimea subarborelui nu se schimbă, dar rădăcina lui da.**

10. Inserare la rădăcină (Root Insertion)

Mecanismul prin care un nod nou ajunge rădăcină păstrând proprietatea BST:

1. Inserezi recursiv z la rădăcina subarborelui potrivit (stânga sau dreapta, după comparație);
2. Rotești ca să-l urci (left-rotate dacă a coborât la dreapta, right-rotate dacă la stânga).


```
Tree-Root-Insert(x, z):
  1 if x = nil
  2   return z
  3 if z.key < x.key
  4   x.left = Tree-Root-Insert(x.left, z)
  5   return Right-Rotate(x)
  6 else x.right = Tree-Root-Insert(x.right, z)
  7   return Left-Rotate(x)
```

11. Strategii pentru cazul defavorabil (motivează cursurile următoare)

🌀 Trei strategii generale de a controla complexitatea în cazul cel mai rău (asta pregătește AVL/Red-Black/Splay):

1. **Randomizare:** transformă orice caz în cazul mediu (cazul rău rămâne posibil, dar extrem de improbabil).
2. **Întreținere amortizată:** ex. rebalansarea unui BST sau redimensionarea unui hash table — operații relativ scumpe dar „amortizate” în timp.
3. **Structuri optimizate (auto-balansate):** garanție $O(\log n)$ în cazul cel mai rău (AVL, Red-Black — vin în cursurile următoare).

🌀 Asta e fix răspunsul la grila2025 Q17: BST-ul simplu **NU** garantează $O(\log n)$ worst-case (poate degenera), dar structurile auto-balansate (AVL, Red-Black, Order-Statistics) **DA**. Splay-ul dă doar amortizat.

12. Ștergere (Deletion) — critic de examen

Trei cazuri, după câți copii are nodul z de șters:

1. **z fără copii (frunză):** îl ștergi pur și simplu.
2. **z cu un copil:** îl ștergi și conectezi $(z.parent)$ la copilul lui z ($(z.right)$ sau $(z.left)$).
3. **z cu doi copii (cazul interesant — delete by copy):**
 - găsești $(y = Tree-Successor(z))$ (= minimul subarborelui drept, care are **cel mult un copil!**);
 - copiezi cheia lui y în z ($(z.key = y.key)$, plus celelalte date);
 - ștergi nodul y (care, având ≤ 1 copil, se șterge ușor prin cazul 1 sau 2).

```

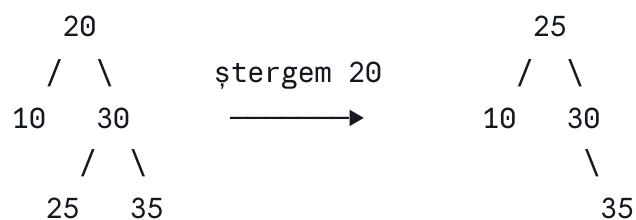
Tree-Delete(T, z):
  1  if z.left = nil or z.right = nil    // z are ≤ 1 copil → îl scoatem
    direct
  2      y = z
  3  else y = Tree-Successor(z)          // z are 2 copii → scoatem succesorul
  4  if y.left ≠ nil
  5      x = y.left
  6  else x = y.right                    // x = singurul copil al lui y (sau
nil)
  7  if x ≠ nil
  8      x.parent = y.parent
  9  if y.parent == nil
  10     T.root = x
  11 else if y == y.parent.left
  12     y.parent.left = x
  13 else y.parent.right = x
  14 if y ≠ z                            // dacă am scos succesorul, copiem
cheia
  15     z.key = y.key
  16     copiază orice alte date din y în z

```

★ Mecanica „delete by copy” (de memorat exact)

🌀 La un nod cu doi copii: **găsești succesorul → copiezi valoarea succesoriului în nodul de șters → elimini fizic nodul succesori** (care are cel mult un copil, deci e ușor). Nu muți nodul z, ci doar îi suprascrii cheia cu cea a succesoriului și ștergi succesoriul.

Exemplu concret:



Succesorul lui 20 = min(subarbore drept) = 25 (mergi dreapta la 30, stânga la 25; 25 n-are copil stâng). Copiezi 25 în rădăcină, ștergi nodul 25 (frunză).

⚠️ Capcana „întotdeauna” (grila2025 Q3 → răspuns (d))

Întrebarea „când facem delete by copy trebuie ÎNTOTDEAUNA să...” are răspunsul „niciuna” pentru că: (1) căutarea succesoriului și copierea se fac **doar** când nodul are doi copii (nu „întotdeauna” — frunzele și nodurile cu un copil se scot direct); (2) poți folosi la fel

de bine **predecesorul** în loc de succesor, deci nu ești *obligat* să folosești succesorul. Fiecare afirmație cu „întotdeauna” pică pe una din aceste nuanțe.

13. Cheat-sheet — fapte și complexități de examen

Concept	De reținut
Proprietatea BST	$\text{stânga} \leq x \leq \text{dreapta}$
Inordine	sortat crescător (Stânga-Vârf-Dreapta)
Reverse-order	sortat descrescător (Dreapta-Vârf-Stânga)
Preordine	Vârf-Stânga-Dreapta
Postordine	Stânga-Dreapta-Vârf; evaluează expresii (postfix)
Complexitate parcurgeri	$\Theta(n)$, optim (ieșirea e $\Theta(n)$)
Minim / Maxim	cel mai din stânga / dreapta
Succesor (cu subarbore drept)	= min subarbore drept = fără copil stâng (frunză sau doar copil drept)
Succesor (fără subarbore drept)	primul strămoș al cărui subarbore stâng îl conține pe x
Căutare / Inserare	$\Theta(h)$; $h = O(\log n)$ mediu, $O(n)$ worst (input sortat)
Inserare	cauți, adaugi unde s-a oprit căutarea (ca frunză)
Rotație	păstrează inordinea + mulțimea nodurilor; schimbă rădăcina; $O(1)$
Ștergere — 0 copii	scoate nodul
Ștergere — 1 copil	leagă părintele de copil
Ștergere — 2 copii	copiază cheia succesorului în z, șterge succesorul
Garanție $O(\log n)$ worst-case	BST simplu NU ; auto-balansat (AVL/RB) DA ; Splay = amortizat
Strategii worst-case	randomizare / amortizat / auto-balansat

14. Conexiuni de examen, pas cu pas

Reconstrucție din parcurgeri (grila2025 Q2, model Q2): în ordinea unui BST e mereu sortată, deci din pre- sau postordine reconstruiești unic arborele (preordinea îți dă rădăcina prima; postordinea ultima), apoi tipărești parcurgerea cerută.

Al k-lea element / median (grila2025 Q13, model Q12): faci în ordine (= sortat) și iei al k-lea / cel din mijloc. Ex: arborele F,B,G,A,D,C,E,I,H → în ordine A,B,C,D,E,F,G,H,I → al 7-lea = G, medianul = E.

Inserare → înălțime (grila2023 Q5): input aproape sortat (1,3,5,10,15,...) → fiecare element merge tot la dreapta → lanț → înălțime $O(n)$. De-asta răspunsul e mare (5), nu $O(\log n)$.

Inserare → postordine (grila2023 Q25): construiești BST-ul inserând în ordinea dată (fiecare element coboară pe drumul de căutare), apoi citești postordinea.

Ștergere prin copiere (grila2025 Q3, model Q3): vezi §12 — succesor = fără copil stâng; copiezi valoarea succesorului în nodul șters; „întotdeauna” e capcană.

rotații (model Q7, grila2023 Q10): după „rotește 6 în jurul lui 3”, aplici invariantul — aceleași noduri, rădăcina subarborelui se schimbă din 3 în 6, copiii se reatașează conform pseudocodului.

Bottom line: ăsta e cursul-cheie al examenului. Memorează: (1) în ordine = crescător, reverse-order = descrescător, postordine = evaluare expresii; (2) succesorul cu subarbore drept = min-ul lui = fără copil stâng (frunză sau doar copil drept) — capcana „fără copil drept”; (3) inserarea coboară pe drumul de căutare și adaugă ca frunză → input sortat dă înălțime $O(n)$; (4) rotația păstrează în ordine și mulțimea nodurilor dar schimbă rădăcina, e $O(1)$; (5) ștergerea cu doi copii = copiezi cheia succesorului în nod și ștergi succesorul. Plus: BST simplu NU garantează $O(\log n)$ worst-case — asta motivează AVL/Red-Black din cursurile următoare, unde randomizarea/balansarea/auto-balansarea rezolvă problema înălțimii.